



TXW80X USB Solution Development Guide



Notice

Due to product version upgrades or other reasons, this document will be updated from time to time. Unless otherwise agreed, this document only

As a guide for use without any warranty.

Zhuhai Taixin
Semiconductor Co.,Ltd

3rd Floor, Building 11, Science and Technology Innovation Park, No. 1, Gangwan, High-tech Zone, Zhuhai City

Confidentiality level	A	TXW80X USB Solution Development Guide	File No	TX-0000
Release date	2022-05-25		file version	V1.0

Revision history

date	Version	describe	Revised by
2022-05-25	V1.0	initial version	TX

	Zhuhai Taixin Semiconductor Co., Ltd. Zhuhai Taixin Semiconductor Co.,Ltd	3rd Floor, Building 11, Science and Technology Innovation Park, No. 1, Gangwan, High-tech Zone, Zhuhai City
---	--	---

Copyright infringement will be investigated

Copyright © 2022 by Tai Xin All rights reserved

Confidentiality level	A	TXW80X USB Solution Development Guide	File No	TX-0000
Release date 2022-05-25			file version	V1.0
Table of contents TXW80X USB solution development guide..... 1 1 Overview..... 1 2. USB Common..... 1 3. USB HOST..... 3 3.1. USB Host related configuration..... 3 3.2. USB Host workflow..... 3 4. USB Device..... 4 4.1. USB Device related configuration.....4 4.2. USB Device workflow.....7				
		Zhuhai Taixin Semiconductor Co., Ltd. Zhuhai Taixin Semiconductor Co.,Ltd	3rd Floor, Building 11, Science and Technology Innovation Park, No. 1, Gangwan, High-tech Zone, Zhuhai City	
Copyright infringement will be investigated Copyright © 2022 by Tai Xin All rights reserved				

1 Overview

This article is written for program software design developers to help you quickly get started with USB program development.

This document is mainly applicable to the following

engineers: y Technical support

engineers y Solution software development engineers

Product scope applicable to this document:

model	encapsulation	Package
TXW806		
TXW803		
TXW802		
TXW801		

2. USB Common

TXW80X USB supports DEVICE and HOST modes, supports USB2.0 HighSpeed transmission, and has EP0/1/2 in total.

There are three transceiver endpoints. In addition to EP0 enumeration, there are four transceiver endpoints, EP1/2.

Please see usb_device.h for the USB device hardware abstraction layer. This abstraction includes USB HOST and DEVICE.

```
69: struct usb_device {
70:     struct dev_obj dev;
71:     uint32 obj;
72:     int32 (*open)(struct usb_device *p_usb_d, struct usb_device_cfg *p_usbdev_cfg);
73:     int32 (*close)(struct usb_device *p_usb_d);
74:     int32 (*read)(struct usb_device *p_usb_d, int8 ep, int8 *buff, uint32 len, int8 sync);
75:     int32 (*write)(struct usb_device *p_usb_d, int8 ep, int8 *buff, uint32 len, int8 sync);
76:     int32 (*ioctl)(struct usb_device *p_usb_d, uint32 cmd, uint32 param1, uint32 param2);
77:     int32 (*request_irq)(struct usb_device *p_usb_d, usbdev_irq_hdl irqhdl, uint32 data);
78: };
79:
80: int32 usb_device_open(struct usb_device *p_usb_d, struct usb_device_cfg *p_usbdev_cfg);
81:
82: int32 usb_device_close(struct usb_device *p_usb_d);
83:
84: int32 usb_device_write(struct usb_device *p_usb_d, int8 ep, int8 *buff, uint32 len, int8 sync);
85:
86: int32 usb_device_read(struct usb_device *p_usb_d, int8 ep, int8 *buff, uint32 len, int8 sync);
87:
88: int32 usb_device_ioctl(struct usb_device *p_usb_d, uint32 cmd, uint32 param1, uint32 param2);
89:
90: int32 usb_device_request_irq(struct usb_device *p_usb_d, usbdev_irq_hdl handle, uint32 data);
91:
```

The usage steps are as follows:

1. Like other peripherals, add the device to the system device list in device.c before use.

```

351:
352: #if USB_EN
353: #if USB_HOST_EN
354:     ... hghostb20_host_attach(HG_USBDEV_DEVID, &usb20_dev);
355: #else
356:     ... hghostb20_dev_attach(HG_USBDEV_DEVID, &usb20_dev);
357: #endif
358: #endif
359:

```

, get the device handle (dev_get), open the device (usb_device_open), configure device 2 (usb_device_ioctl), interrupt request (usb_device_request_irq)

```

196: ... usb_test.dev = (struct usb_device *) dev_get(HG_USBDEV_DEVID);
197: ... if (usb_test.dev) {
198:     ... usb_host_task_init(usb_test.dev);
199:     ... if (!usb_device_open(usb_test.dev, NULL)) {
200:         ... usb_host_uvc_ioctl(usb_test.dev, USB_HOST_IO_CMD_INSERT_HUB, &msgbuf, 0);
201:         ... printf("%s:%d\n", __FUNCTION__, __LINE__);
202:         ... uint32 usb_host_bus_irq(uint32 irq, uint32 param1, uint32 param2, uint32 param3);
203:         ... usb_device_request_irq(usb_test.dev, usb_host_bus_irq, (int32)usb_test.dev);
204:     }
205: }

```

Detailed

description: 1. usb_device_ioctl usage instructions

```

24: enum usb_dev_io_cmd {
25:     ... USB_DEV_IO_CMD_AUTO_TX_NULL_PKT_ENABLE,
26:     ... USB_DEV_IO_CMD_AUTO_TX_NULL_PKT_DISABLE,
27:     ... USB_DEV_IO_CMD_REMOTE_WAKEUP,
28:
29:     ... /* this function need call after attach & before open
30:        ... * msg[1:0]:: vid
31:        ... * msg[3:2]:: pid
32:        ... */
33:     ... USB_DEV_IO_CMD_SET_ID,
34: };
35:

```

ÿUSB_DEV_IO_CMD_AUTO_TX_NULL_PKT_ENABLEÿÿ

[USB_DEV_IO_CMD_AUTO_TX_NULL_PKT_DISABLE]: USB automatically sends empty packets: when sending the packet length and

When MaxPacketSize is aligned, empty packets are automatically reissued as short packet end switches.

[USB_DEV_IO_CMD_REMOTE_WAKEUP]: USB Device remotely wakes up Host

[USB_DEV_IO_CMD_SET_ID]: USB Device VID/PID setting, this action needs to be executed immediately after usb_device_open.

2. usb_device_request_irq usage instructions

The currently supported USB IRQs are as follows:

```

9:
10: enum usb_dev_irqs {
11:     ... USB_DEV_RESET_IRQ,
12:     ... USB_DEV_SUSPEND_IRQ,
13:     ... USB_DEV_RESUME_IRQ,
14:     ... USB_DEV_SOF_IRQ,
15:     ... USB_DEV_CTL_IRQ,
16:     ... USB_EP_RX_IRQ,
17:     ... USB_EP_TX_IRQ,
18:     ... USB_CONNECT,
19:     ... USB_DISCONNECT,
20:     ... USB_BABBLE,
21:     ... USB_XACT_ERR,
22: };
23:

```

[USB_DEV_RESET_IRQ]: usb device receives bus reset signal

[USB_DEV_SUSPEND_IRQ]: usb device receives bus suspend signal

[USB_DEV_RESUME_IRQ]: usb device receives bus resume signal

[USB_DEV_SOF_IRQ]: usb device receives bus sof signal

[USB_DEV_CTL_IRQ]: usb endpoint 0 transceiver interrupt

[USB_EP_RX_IRQ]: USB endpoint receives data interrupt

[USB_EP_TX_IRQ]: USB endpoint completes sending data interrupt

[USB_CONNECT]: The usb host found that the device was connected and interrupted.

[USB_DISCONNECT]: usb host found that the device connection was disconnected and interrupted.

[USB_BABBLE]: usb host encounters babble

[USB_XACT_ERR]: usb host encountered xact error

3. USB HOST

3.1. USB Host related configuration

For the solution configuration, see project_config.h configuration: USB_HOST_EN and USB_EN are both enabled.

3.2. USB Host workflow

USB Host software configuration and workflow are as follows.

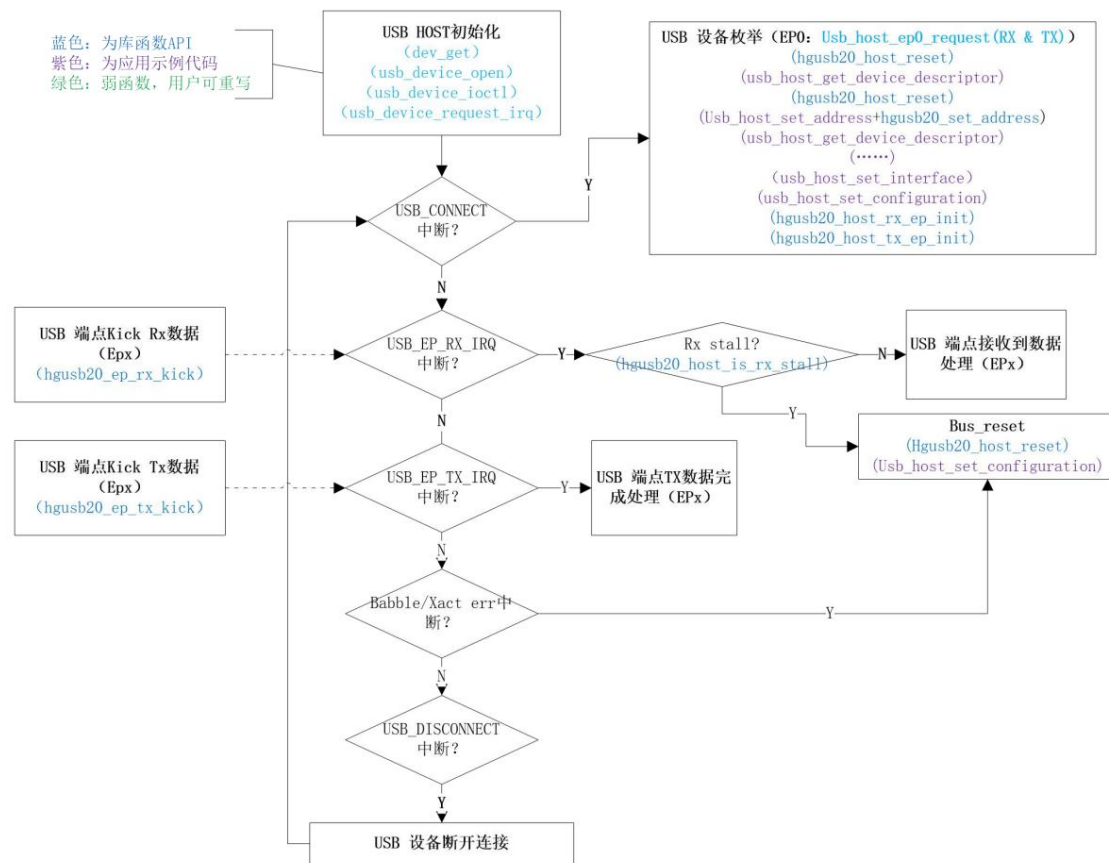


Figure 3.2.1 USB Host workflow chart

4. USB Device

4.1. USB Device related configuration

For the solution configuration, see project_config.h configuration: USB_EN enabled.

Device configuration has been abstracted into a structure: struct usb_device_cfg. This structure is in

When usb_device_open() is passed in, the corresponding information will be extracted for enumeration during USB background enumeration. The structure is defined as follows

Down:

```

44: struct usb_device_cfg {
45:     ...uint16_t vid;
46:     ...uint16_t pid;
47:     ...uint8_t speed;
48:     ...uint8_t *p_device_descriptor;
49:     ...uint8_t *p_config_descriptor_head;
50:     ...void *p_config_desc;
51:     ...uint16_t config_desc_len;
52:     ...uint8_t interface_num;
53:
54:     ...uint8_t *p_language_id;
55:     ...uint16_t language_id_len;
56:     ...uint8_t *p_str_manufacturer;
57:     ...uint16_t str_manufacturer_len;
58:     ...uint8_t *p_str_product;
59:     ...uint16_t str_product_len;
60:     ...uint8_t *p_str_serial_number;
61:     ...uint16_t str_serial_number_len;
62:
63:     ...uint8_t ep_nums;
64:     ...struct usb_device_ep_cfg ep_cfg[8]; /* must same with config_desc */
65: } __attribute__((packed));
66:

```

vid: Manufacturer ID

[pid]: Product ID

speed: Not used yet

[p_device_descriptor]: device descriptor

[p_config_descriptor_head]: Configuration descriptor general header information

[p_config_desc]: Configuration descriptor

[config_desc_len]: Configuration descriptor length

[interface_num]: Number of interfaces

[p_language_id]: Language ID descriptor

[language_id_len]: Language ID descriptor length

[p_str_manufacturer]: manufacturer descriptor

[str_manufacturer_len]: Manufacturer descriptor length

[p_str_product]: product descriptor

[str_product_len]: product descriptor length

[p_str_serial_number]: Serial number descriptor (can be empty if not available)

[str_serial_number_len]: Serial number descriptor length

[ep_nums]: Number of endpoints (EP0 is not included)

[ep_cfg]: Endpoint configuration: endpoint number, transmission direction, transmission type, maximum packet size

The following is a configuration example of a custom USB device class and USB Audio class:


```

153:
154: static const struct usb_device_cfg usb_dev_wifi_cfg = {
155:     .vid = USB_VID,
156:     .pid = USB_PID,
157:     .speed = USB_SPEED_HIGH,
158:     .p_device_descriptor = (uint8_t*)tbl_usb_wifi_device_descriptor,
159:     .p_config_descriptor_head = (uint8_t*)tbl_usb_wifi_config_all_descriptor,
160:
161:     .p_config_desc = (uint8_t*)tbl_usb_wifi_config_all_descriptor_wifi,
162:     .config_desc_len = sizeof(tbl_usb_wifi_config_all_descriptor_wifi),
163:     .interface_num = 1,
164:
165:     .p_language_id = (uint8_t*)tbl_usb_wifi_language_id,
166:     .language_id_len = sizeof(tbl_usb_wifi_language_id),
167:     .p_str_manufacturer = (uint8_t*)tbl_usb_wifi_str_manufacturer,
168:     .str_manufacturer_len = sizeof(tbl_usb_wifi_str_manufacturer),
169:     .p_str_product = (uint8_t*)tbl_usb_wifi_str_product,
170:     .str_product_len = sizeof(tbl_usb_wifi_str_product),
171:     .p_str_serial_number = (uint8_t*)tbl_usb_wifi_str_serial_number,
172:     .str_serial_number_len = sizeof(tbl_usb_wifi_str_serial_number),
173:
174:     .ep_nums = 2,
175:     .ep_cfg[0].ep_id = USB_WIFI_RX_EP,
176:     .ep_cfg[0].ep_type = USB_ENDPOINT_XFER_BULK,
177:     .ep_cfg[0].ep_dir_tx = 0,
178:     .ep_cfg[0].max_packet_size = USB_WIFI_EP_MAX_PKT_SIZE,
179:     .ep_cfg[1].ep_id = USB_WIFI_TX_EP,
180:     .ep_cfg[1].ep_type = USB_ENDPOINT_XFER_BULK,
181:     .ep_cfg[1].ep_dir_tx = 1,
182:     .ep_cfg[1].max_packet_size = USB_WIFI_EP_MAX_PKT_SIZE,
183: };
184:

```

Figure 4.1.1 Custom USB device class configuration example

```

389: const struct usb_device_cfg usb_dev_audio_cfg = {
390:     .vid = USB_VID,
391:     .pid = USB_AUDIO_PID,
392:     .speed = USB_SPEED_HIGH,
393:     .p_device_descriptor = (uint8_t*)tbl_usb_audio_device_descriptor,
394:     .p_config_descriptor_head = (uint8_t*)tbl_usb_audio_config_all_descriptor_gen,
395:     .p_config_desc = (uint8_t*)tbl_usb_audio_config_all_descriptor_audio,
396:     .config_desc_len = sizeof(tbl_usb_audio_config_all_descriptor_audio),
397:     .interface_num = USB_AUDIO_INTERFACE_NUMS+1,
398:     .p_language_id = (uint8_t*)tbl_usb_audio_language_id,
399:     .language_id_len = sizeof(tbl_usb_audio_language_id),
400:     .p_str_manufacturer = (uint8_t*)tbl_usb_audio_str_manufacturer,
401:     .str_manufacturer_len = sizeof(tbl_usb_audio_str_manufacturer),
402:     .p_str_product = (uint8_t*)tbl_usb_audio_str_product,
403:     .str_product_len = sizeof(tbl_usb_audio_str_product),
404:     .p_str_serial_number = (uint8_t*)tbl_usb_audio_str_serial_number,
405:     .str_serial_number_len = sizeof(tbl_usb_audio_str_serial_number),
406:     .ep_nums = USB_AUDIO_INTERFACE_NUMS,
407:     .ep_cfg[0].ep_id = USB_AUDIO_SPK_EP,
408:     .ep_cfg[0].ep_type = USB_ENDPOINT_XFER_ISOC,
409:     .ep_cfg[0].ep_dir_tx = 0,
410:     .ep_cfg[0].max_packet_size = USB_AUDIO_SPK_MAX_PKT_SIZE,
411:     .ep_cfg[1].ep_id = USB_AUDIO_MIC_EP,
412:     .ep_cfg[1].ep_type = USB_ENDPOINT_XFER_ISOC,
413:     .ep_cfg[1].ep_dir_tx = 1,
414:     .ep_cfg[1].max_packet_size = USB_AUDIO_MIC_MAX_PKT_SIZE,
415:     .ep_cfg[0].ep_id = USB_AUDIO_SPK_EP,
416:     .ep_cfg[0].ep_type = USB_ENDPOINT_XFER_ISOC,
417:     .ep_cfg[0].ep_dir_tx = 0,
418:     .ep_cfg[0].max_packet_size = USB_AUDIO_SPK_MAX_PKT_SIZE,
419:     .ep_cfg[1].ep_id = USB_AUDIO_MIC_EP,
420:     .ep_cfg[1].ep_type = USB_ENDPOINT_XFER_ISOC,
421:     .ep_cfg[1].ep_dir_tx = 1,
422:     .ep_cfg[1].max_packet_size = USB_AUDIO_MIC_MAX_PKT_SIZE,
423:     .ep_cfg[0].ep_id = USB_AUDIO_SPK_EP,
424:     .ep_cfg[0].ep_type = USB_ENDPOINT_XFER_ISOC,
425:     .ep_cfg[0].ep_dir_tx = 0,
426:     .ep_cfg[0].max_packet_size = USB_AUDIO_SPK_MAX_PKT_SIZE,
427:     .ep_cfg[1].ep_id = USB_AUDIO_MIC_EP,
428:     .ep_cfg[1].ep_type = USB_ENDPOINT_XFER_ISOC,
429:     .ep_cfg[1].ep_dir_tx = 1,
430:     .ep_cfg[1].max_packet_size = USB_AUDIO_MIC_MAX_PKT_SIZE,
431: };
432:

```

Figure 4.1.2 USB Audio class configuration example

4.2. USB Device workflow

USB Device configuration and workflow such as:

Figure 4.2.1 USB Device software configuration and main work flow

Figure 4.2.2 USB Device EP0 background enumeration process

Under normal circumstances, for a simple USB Device solution, the user only needs to configure struct usb_device_cfg

can be realized;

If configuring usb_device_cfg cannot meet user needs, you can rewrite the function implementation in the green part.

The function that the user wants to implement requires familiarity with the USB protocol. It is not recommended to use it unless necessary.

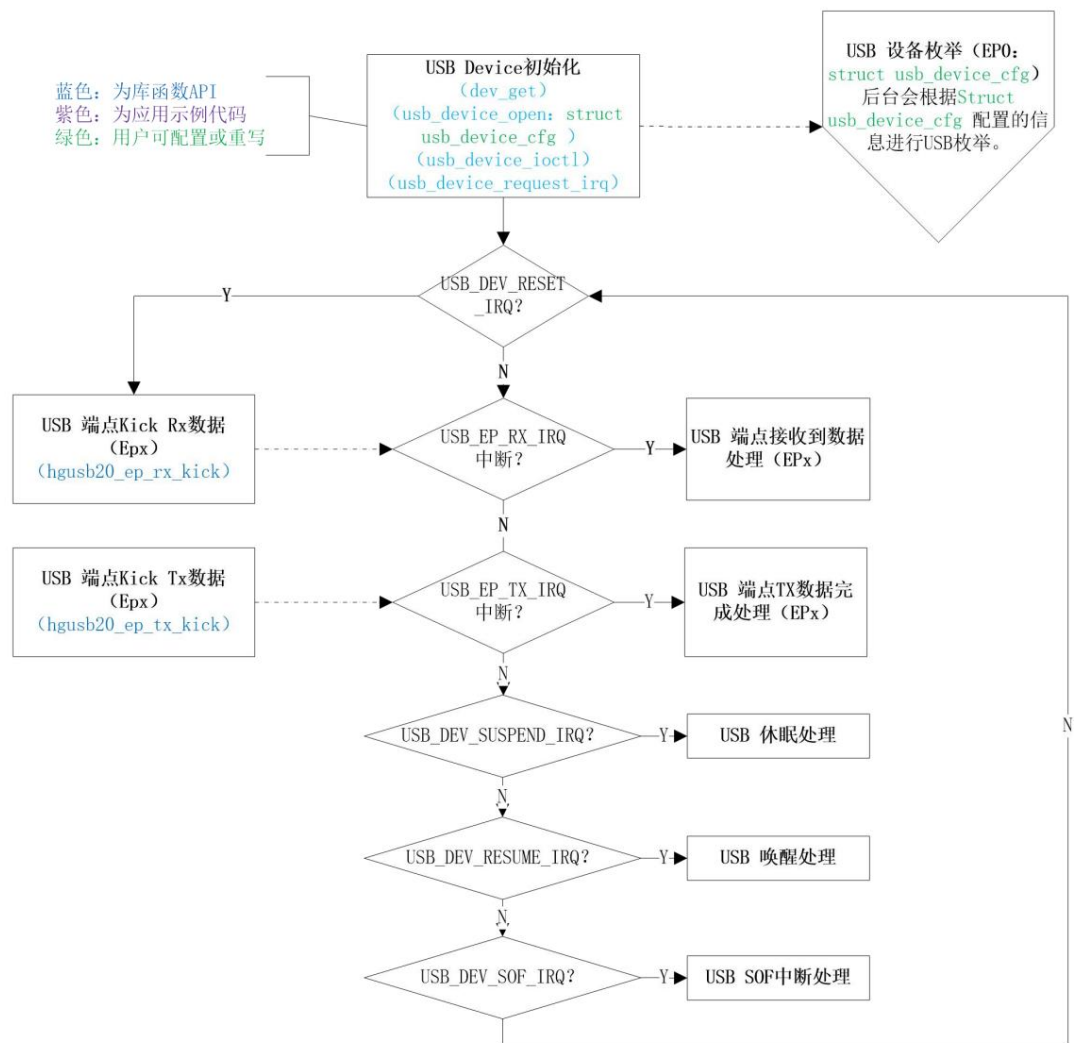


Figure 4.2.1 USB Device software configuration and main work flow

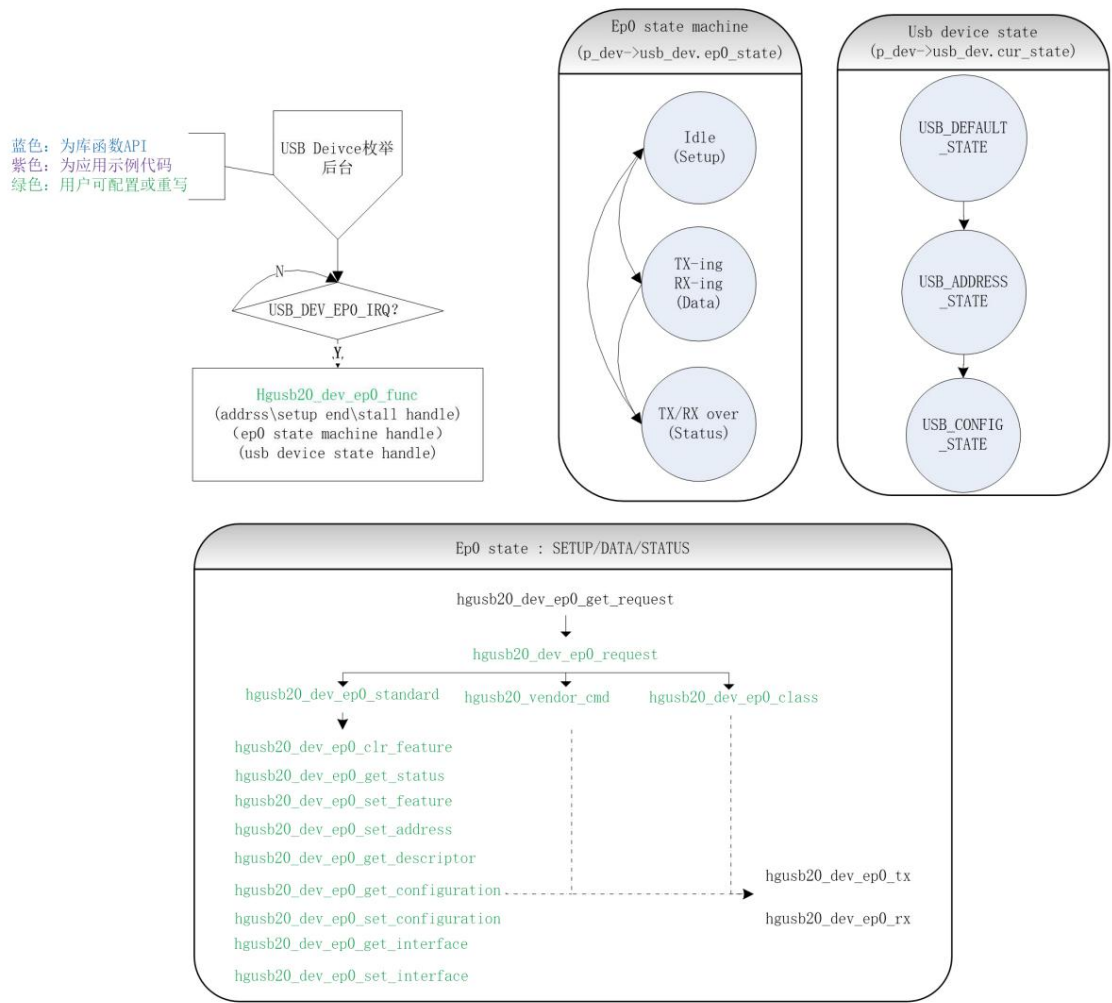


Figure 4.2.2 USB Device EP0 background enumeration process